
Consumo de APIs en Next.js

TanStack Query · React Hook Form · Zod

Por qué se aplican: datos con caché, formularios validados y estado del cliente, sin reinventar la rueda.

Agenda

Qué vamos a ver y por qué

- **El problema actual:** cómo conectamos el frontend a la API hoy (fetch manual, mock, estados repetidos).
- **Server State vs Client State:** dos tipos de estado que no se deben mezclar.
- **TanStack Query:** useQuery, useMutation, caché, invalidación y optimistic updates.
- **React Hook Form + Zod:** formularios gestionados con validación tipada en un solo esquema.
- **Zustand:** el estado del navegador (carrito, filtros) sin fricción.
- **Estados de UI:** loading, error y empty de una sola forma en todo el sistema.
- **Proyecto ejemplo** en Material-Clase/react-query-rhf-example y cómo aplicarlo a cada grupo.

El problema: cómo conectamos hoy

Lo que se repite en los proyectos y por qué hay que cambiarlo

- **fetch() suelto en cada componente** → la lógica de datos queda dispersa y sin tipado.
- **Estados manuales con useEffect + useState** → cada pantalla reinventa loading/error/vacío (y se olvida de alguno).
- **Fallback a mock data** → la app funciona con datos falsos y la integración real queda sin probar.
- **Sin caché** → cada navegación vuelve a pedir la lista entera al backend.
- **Validación duplicada** → reglas en el HTML, en el submit y en el backend, sin una fuente única.

Estos tres problemas se resuelven con las tres librerías del taller.

Server State vs Client State

El concepto base: hay dos tipos de estado y no se deben mezclar

- **Server State (estado de servidor):** los datos que viven en la API (productos, usuarios, vehículos...).
- Es asíncrono, compartido, y puede quedar desactualizado si otro usuario cambió los datos.
- **Client State (estado de cliente):** lo que solo existe en el navegador (carrito, filtro activo, tema, menú abierto).
- **Mezclarlos** es la causa de bugs clásicos: datos que no se actualizan o UI que no refleja la API.
- **La división de trabajo:** TanStack Query → server state · Zustand → client state.

TanStack Query — qué es y por qué

La capa de datos del frontend

- **Librería estándar para datos asíncronos en React** (fetching, caché, sincronización y actualizaciones).
- **useQuery** para leer (GET) con caché y estados listos.
- **useMutation** para escribir (POST/PUT/DELETE) con loading, error y rollback.
- **Invalidación de caché:** tras una mutación, la lista se vuelve a pedir automáticamente.
- **Por qué:** deja de esconder la integración detrás de mocks y resuelve los estados de design.md en una línea.

```
const { data, isLoading, isError, refetch } = useQuery({  
  queryKey: ['productos'],  
  queryFn: () => apiRequest('/productos'),  
});
```

useQuery — los estados resueltos

Cada pantalla de listado debería contemplar: carga, error y vacío

- **isLoading:** primer pedido en curso → skeleton.
 - **isError + error.message:** fallo de red o 404/500 → mensaje claro + botón reintentar.
 - **data vacía:** 0 resultados → empty state que invita a crear.
 - **data con contenido:** la lista, servida desde la caché si ya se pidió antes.
-
- **Por qué:** es el mismo patrón en todas las entidades (consistencia) y el código de estados no se duplica.

En el proyecto ejemplo esto está en `app/productos/page.tsx` (`LoadingSkeleton`, `ErrorState`, `EmptyState`).

useMutation + invalidación de caché

Escribir en la API y mantener la UI sincronizada

- **useMutation** maneja el ciclo de una escritura: `isPending`, `isError`, `onSuccess`.
- **Al crear/editar un producto:** `onSuccess` → `invalidateQueries(['productos'])`.
- **invalidateQueries** marca la lista como vieja y la vuelve a pedir: la UI refleja la API.

```
const crear = useMutation({
  mutationFn: (datos) => apiRequest('/productos', { method: 'POST', body:
JSON.stringify(datos) }),
  onSuccess: () => queryClient.invalidateQueries({ queryKey: ['productos'] }),
});
```

- **Por qué:** sin esto, crear un elemento no aparecería en el listado hasta recargar la página.

Actualización optimista (optimistic updates)

UI instantánea con rollback si el servidor falla

- **onMutate:** modificamos la caché al instante (el elemento desaparece de la UI ya).
- **onError:** si la API rechaza, restauramos el estado anterior (rollback).
- **onSettled:** al final siempre se reconcilia contra el servidor (invalidate).

```
const eliminar = useMutation({
  mutationFn: (id) => apiRequest(`/productos/${id}`, { method: 'DELETE' }),
  onMutate: (id) => { cancelarQueries(); quitarDeLaCache(id); return { anterior }; },
  onError: (_e, _id, ctx) => restaurar(ctx.anterior),
  onSettled: () => invalidar(),
});
```

- **Por qué:** la app se siente rápida y, si falla, vuelve sola al estado correcto.

React Hook Form — por qué

Formularios gestionados sin estado manual

- **Maneja valores, errores por campo, touched y submit** sin useState por input.
- **Registro declarativo:** register('nombre') + valueAsNumber para los números.
- **Menos re-renders:** solo se actualiza lo que cambia.
- **Deshabilita el botón mientras se envía (isPending)** y cancela el submit si ya está en curso.
- **Por qué:** los formularios son la parte con más bugs; gestionarlos de una forma consistente los vuelve predecibles.

Componente ProductoForm.tsx: el mismo formulario sirve para alta y edición.

Zod — validación tipada compartida

Las reglas y los tipos en un solo lugar

- **Esquema = fuente única de verdad:** reglas (min, max, positive, int) y tipos inferidos (z.infer).
- **z.number({ message })** da mensajes en español por campo.
- **Tipa el submit:** los valores del formulario llegan tipados a la mutación.

```
export const productoFormSchema = z.object({
  nombre: z.string().min(3, 'El nombre debe tener al menos 3 caracteres'),
  precio: z.number({ message: 'Ingresá un precio válido' }).positive(),
  stock: z.number().int().nonnegative(),
});
export type ProductoFormValues = z.infer<typeof productoFormSchema>;
```

- **Por qué:** la validación deja de estar duplicada en HTML + JS + backend y el tipado acompaña al dato.

React Hook Form + Zod (el puente)

zodResolver conecta el esquema con el formulario

- **El resolver valida en cada cambio y alimenta formState.errors** con los mensajes de Zod.
- **Los errores se muestran por campo** debajo de cada input, no en un bloque genérico.

```
const { register, handleSubmit, formState: { errors } } =  
  useForm<ProductoFormValues>({  
    resolver: zodResolver(productoFormSchema),  
    defaultValues, // precargado en edición  
  });
```

```
<form onSubmit={handleSubmit(onSubmit)}>  
  <input {...register('nombre')} />  
  {errors.nombre && <p>{errors.nombre.message}</p>}  
</form>
```

- **Por qué:** la misma validación tipada que define el contrato es la que corre en el navegador.

Zustand — estado del cliente

Lo que es solo del navegador no debe ir a la API

- **Store global mínimo con hook:** useCarrito → items, agregar, quitar, limpiar.
- **Sin Provider:** se importa el hook y listo.
- **Suscriptores selectivos:** useCarrito((s) ⇒ s.items.length) solo re-renderiza el contador.

```
export const useCarrito = create((set) => ({
  items: [],
  agregar: (id) => set((s) => ({ items: [...s.items, id] })),
  quitar: (id) => set((s) => ({ items: s.items.filter((i) => i !== id) })),
}));
```

```
const cantidad = useCarrito((s) => s.items.length);
```

- **Por qué:** separar client state de server state evita el problema del slide 4.

Estados de UI en la práctica

Lo que pedía design.md: cada pantalla con carga, error, vacío y datos

- **LoadingSkeleton** · **ErrorState con reintentar** · **EmptyState** · datos.
- **Se resuelven una vez en componentes de UI** (components/ui/Estado.tsx) y se reutilizan en todas las entidades.
- **Con useQuery** isLoading / isError / data.length === 0 salen gratis; solo resta mostrarlos.
- **Por qué:** consistencia visual y menos decisiones por pantalla: el patrón se copia, no se reinventa.

Estructura del proyecto ejemplo

Material-Clase/react-query-rhf-example

```
backend/    → API mínima FastAPI con /api/productos (CRUD)
frontend/
  lib/api.ts      cliente HTTP tipado (un solo fetch)
  lib/providers.tsx  QueryClientProvider global
  lib/store.ts    Zustand: carrito (client state)
  schemas/producto.ts  esquema Zod + tipos inferidos
  queries/productos.ts hooks useQuery / useMutation
  components/ProductoForm.tsx  RHF + Zod (alta y edición)
  app/productos/    listado · nuevo · [id]/editar
```

- **Separación por responsabilidad:** datos (queries), contratos (schemas), UI (components) y páginas (app).
- **El frontend consume /api mediante rewrites** de next.config → localhost:8000.

Buenas prácticas y patrones

Lo que se espera ver en los proyectos de los grupos

- **Una capa de servicios/queries** y un cliente HTTP tipado; nunca `fetch()` suelto en componentes.
- **Esquemas Zod como contrato** compartido entre formularios y llamadas a la API.
- **Estados de UI centralizados** (Skeleton / ErrorState / EmptyState) reutilizables.
- **Invalidación después de cada mutación** y optimistic updates solo en acciones críticas (eliminar).
- **Mensajes de error en español por campo** y botón deshabilitado mientras se procesa.

Cómo se aplica a cada proyecto

El tema cierra la brecha de los grupos

- **G2 (Auto Market):** reemplazar MOCK_VEHICLES y filtros locales por useQuery conectado a /api/v1/vehicles.
- **G1 (Cafecito):** migrar los updates optimistas de mesas a useMutation con onMutate/onError.
- **G3 (Restaurante):** ya usa TanStack Query + RHF + Zod: sirve de referencia del patrón.
- **G4 (Agro):** arrancar el frontend con esta estructura desde cero.

El objetivo: los 4 grupos terminan con fetching con caché, formularios validados y estados de UI consistentes.

Para llevar

TanStack Query para los datos · React Hook Form + Zod para los formularios · Zustand para la UI.

Proyecto de práctica: Material-Clase/react-query-rhf-example
Presentación: tanstack-query-rhf.pptx

¡A programar!